Supplementary Materials for "MA-SQLGrid: A Multi-Stage Context-Grounding Framework for Text-to-SQL over Power Grid Maintenance Databases"

Bijing Liu, Chenglong Sun and Yong Yang

This document contains the supplementary materials referenced from the main text: the verbatim prompt templates for the five experimental conditions (Section S1), the per-question case SQL listings for the three case studies (Section S2), the supplementary tables (Section S3, Tables S1–S7), and the scale-robustness experiment details (Section S4, Tables S8–S9). All contents are reproduced from the archived experiment artifacts; no experimental number differs from the released run records.

S1 Prompt Templates for the Five Conditions

All five conditions share fixed prompt templates; only the context block differs. The templates below are reproduced verbatim from the archived experiment runner, and every rendered prompt (with its hash) is included in the released traces. Placeholders in angle brackets are filled per question.

S1.1 Direct-Generation Template (C1–C4)

```
You are a Text-to-SQL system for a synthetic
SQLite power-grid maintenance database.

Return exactly one read-only SQLite SELECT
query. Do not include markdown or explanation.
Do not use INSERT, UPDATE, DELETE, DROP,
PRAGMA, or multiple statements.
Use only the provided database context. When
normalization hints or answer-shape hints are
present, treat them as question-derived
constraints. Match the requested projection
count and include relevant literal predicates.

Condition: <condition name>

<context block>

Question ID: <question id>
Question: <question text>
```

The context block is the only element that varies across conditions: C1 inserts the full raw `schema.sql`; C2 inserts a rendered dump of the full schema plus the enumerated distinct values of all non-ID columns; C3 inserts the generically selected schema subset produced by lexical matching without domain rules, value normalization, or shape hints; C4 inserts the compact domain context shown below.

S1.2 Rendered Compact Domain Context (C4/C5), Example for Q021

```
SQLite selected context:
Tables and selected columns:
- asset_types(asset_type_id) -- equipment type
  catalog such as Transformer, Breaker, Line,
  Relay, Substation, Capacitor
- assets(asset_id, asset_name, asset_type_id,
  location_id, status) -- individual grid
  assets with names, type, location, install
  date, status, and MW capacity
- grid_topology(edge_id, upstream_asset_id,
  downstream_asset_id) -- directed
  upstream/downstream asset connections and
  switch status
- locations(location_id) -- grid yards,
  substations, regions, coordinates, and
  location criticality
- sensor_readings(reading_id, asset_id) --
  time-series sensor readings for assets with
  sensor type, value, unit, and alarm flag
- work_orders(work_order_id, asset_id,
  assigned_technician_id) -- maintenance work
  orders with priority, status, schedule,
  completion date, and fault code
Join paths:
- assets.asset_type_id =
  asset_types.asset_type_id
- assets.location_id = locations.location_id
- work_orders.asset_id = assets.asset_id
- sensor_readings.asset_id = assets.asset_id
- grid_topology.upstream_asset_id =
  assets.asset_id
- grid_topology.downstream_asset_id =
  assets.asset_id
Exact database values matched from the
question:
- assets.status: offline
Power-grid domain normalization hints inferred
from the question:
- assets.status = "offline"
Answer-shape hints inferred from the question
text:
- expected column count: 1
- row granularity: multi-row
- deterministic ordering needed: True
- Project asset_name for asset listing
```

```
questions.
```

S1.3 Candidate-Generation Template (C5)

```
You are generating candidate SQLite SQL for
CHESS-style MA-SQLGrid.

Return 3 distinct read-only SQLite SELECT
queries as a numbered list. Do not include
explanation.
Do not use INSERT, UPDATE, DELETE, DROP,
PRAGMA, or multiple statements.
Use only the selected context, normalization
hints, and inferred output-form hints below.
Candidate queries should differ in plausible
joins/projections while respecting the
inferred projection count and relevant literal
predicates.

Condition: <condition name>

<compact domain context block, as in C4>

Question ID: <question id>
Question: <question text>
```

S1.4 Bounded Repair Template (C5, at Most One Call)

```
Repair this SQLite SELECT query using only
selected context and inferred hints.

Return exactly one read-only SQLite SELECT
query. Do not include markdown or explanation.

<compact domain context block, as in C4>

Question ID: <question id>
Question: <question text>
Previous SQL: <selected candidate SQL>
Reference-free validation result:
<JSON validation record: exec_ok, shape_ok,
order_ok, empty result, matched and missing
value hints>
```

The repair prompt receives only the reference-free validation signal; it never receives gold SQL, gold rows, evaluator verdicts, or dataset metadata.

S2 Case Analysis SQL Listings

Three cases drawn verbatim from the archived traces illustrate the mechanism, the value of validation, and its boundary. These are the full per-question SQL listings summarized in Section 5.6 of the main text.

Case 1 (contract enforcement, Q021). The question asks: "Which assets are offline?" The gold query returns one column, `asset_name`, filtered by `assets.status = 'offline'` and ordered by asset name. The full-schema-values baseline (C2) generates

```
SELECT asset_name FROM assets WHERE status = 'offline';
```

it uses the right predicate but omits the required deterministic ordering and is scored as a denotation mismatch under the order-sensitive evaluator. The compact domain context records the normalized hint `assets.status = "offline"` and an inferred shape with one projected column, multi-row output, and required ordering. Under that context, C4 generates

```
SELECT asset_name FROM assets WHERE status = 'offline' ORDER BY asset_name;
```

while C5 generates the equivalent aliased form. Both match the validated denotation. This is the modal pattern behind the strict-metric gap and is exactly the contract-conformance effect quantified in Section 5.2 of the main text.

Case 2 (execution-guided reranking, Q110). The question asks: "List work orders linked to asset REL-002." The first C5 candidate,

```
SELECT wo.work_order_id, wo.status, wo.schedule
FROM work_orders AS wo
JOIN assets AS a ON wo.asset_id = a.asset_id
WHERE a.asset_name = 'REL-002'
ORDER BY wo.work_order_id;
```

references the nonexistent column `wo.schedule` and fails execution. The reference-free ranker demotes it and selects the second candidate, a subquery formulation projecting `work_order_id`, `priority`, and `status`, which executes and matches the gold denotation. Under C4, which returns only the first formulation, this question is lost. This is the recovery mode that explains most of C5's reduction of execution errors from 20 to 5.

Case 3 (repair success and failure on one question family, Q161 versus Q156/Q157/Q158/Q160/Q162). Six test questions share the template "Which work orders are assigned to technician *X*?" The compact context for these questions selects `work_orders(work_order_id, asset_id, assigned_technician_id, priority, status)` but omits the `scheduled_date` column, while the shape hint asks the model to project `work_order_id`, `status`, and `scheduled_date` and the table comment mentions "schedule." Given this inconsistent evidence, the generator hallucinates `wo.schedule_date` (or `wo.schedule`) in every candidate. For Q161 the single bounded repair call, prompted with the execution error, rewrites the column to the correct `scheduled_date` and the repaired query matches the gold denotation. For the other five questions the repair reproduces the same nonexistent column and is rejected, leaving the execution error in place; these five are exactly the residual C5 execution errors in Table S6. The family shows both faces of bounded repair: it can fix a locally diagnosable naming error, but it cannot reliably recover information that the compact context itself dropped, since the repair prompt reuses that same context. The root cause is a schema-selection miss, which is why the main text attributes the main effect to context construction and treats validation as a bounded second layer.

S3 Supplementary Tables

Table S1: Implementation details fixed in the reported MA-SQLGrid run.

Component	Fixed implementation detail
Schema selection	Lexical question/schema matching, domain phrase rules, exact value mentions, and foreign-key expansion over selected seed tables with a maximum of six tables.
Value inventory	Distinct local database values are enumerated for non-ID categorical/date/status columns; matched literals and fixed domain-normalization rules are rendered as compact hints.
Answer-shape inference	Rule classes infer projection count, row granularity, count/group/top-k/latest/list forms, and deterministic ordering cues from the question text. A post-generation held-out diagnostic found 0 column-count mismatches over 180 test questions; this audit did not feed prompts, rankers, repairs, or candidate selection.
C5 candidate policy	At most five SQL candidates are generated at temperature 0; one bounded repair attempt is allowed only after reference-free validation flags execution, shape, value, or ordering problems.
C5 ranker weights	Safe SQL $+10/-20$, execution $+10/-15$, shape $+6/-5$, ordering $+3/-2$, empty result -2, value hit $+4$, missing value hint -3, and deterministic tie-break $-$candidate index.
No-gold contract	Prediction records exclude gold SQL, gold denotation rows, evaluator correctness, expected hashes, required-literal metadata, answer-shape metadata, and order-sensitive metadata.

Table S2: Dataset characterization for GridDB-Maintenance-v2 v0.1.

Aspect	Observed value
Total question-SQL pairs	200
Dev/test split	Q001–Q020 dev, Q021–Q200 test
Test questions used in formal results	180
Database tables	8
Main entity/value tables	assets; asset types; locations; work orders; sensor readings; technicians; topology; maintenance logs
Difficulty labels	74 easy, 102 medium, 24 hard
Common SQL feature tags	filter 187, order-by 131, join 113, aggregation 66, time predicate 33
Main value vocabularies	asset type, status, region, priority, fault code, sensor type, topology switch state

Table S3: Paired comparisons on the test split. A-only and B-only count questions answered correctly by only one condition.

Comparison	A-only	B-only	Both correct	Both wrong	Mean difference	Exact sign p
C4 vs C2	49	2	77	52	0.2611	1.179e-12
C5 vs C2	56	4	75	45	0.2889	9.085e-13
C5 vs C4	12	7	119	42	0.0278	0.3593

Table S4: Measured resource consumption per question for the `deepseek-chat` second-generator run (provider-reported usage; direct endpoint with negligible fixed serving overhead). C5 combines candidate generation and, when triggered, one repair call.

Condition	Input tokens (mean)	Output tokens (mean)	Latency ms (mean)	Latency ms (median)	First-attempt calls
C2	2011.6	49.9	1051.9	1021.0	180/180
C4	509.3	46.6	1054.0	1003.5	180/180
C5	680.5	186.2	3014.2	1965.5	178/180

Table S5: Execution accuracy by question tag using existing formal-run logs. Rows are overlapping diagnostic subsets.

Tag	Test questions	C2	C4	C5	C4–C2
Join	102	0.382	0.598	0.657	+0.216
Aggregation	59	0.847	0.847	0.831	+0.000
Order-by	114	0.132	0.535	0.596	+0.404
Top-k	18	0.000	0.889	0.833	+0.889
Time predicate	30	0.000	0.533	0.567	+0.533
Group-by	12	0.417	0.333	0.417	−0.083
Topology traversal	11	0.091	0.818	0.909	+0.727
Self-join	9	0.111	0.889	1.000	+0.778
Filter	170	0.441	0.718	0.753	+0.276

Table S6: Residual error taxonomy from the archived formal run. Counts are over 180 test questions. The taxonomy assigns each incorrect prediction to a primary evaluator error category; it is not the same object as the answer-shape accuracy metric, and the execution-error column does not contradict the safe-SQL rate (those queries pass the read-only guard but fail when executed against the SQLite schema).

Condition	Correct	Shape mismatch	Wrong denotation	Execution error
C1	71	95	14	0
C2	79	90	11	0
C3	72	67	26	15
C4	126	0	34	20
C5	131	0	44	5

Table S7: Claim-to-evidence map for the reported MA-SQLGrid results.

Claim	Supporting evidence	Boundary
Compact domain context is the main mechanism gain under the strict answer-contract metric	C4 reaches 126/180 and 0.7000 accuracy; C4 vs C2 has 49 C4-only correct cases, 2 C2-only cases, and +0.2611 mean difference	Applies to GridDB-Maintenance-v2 v0.1 under the fixed generator and the single archived formal pass; the sensitivity analysis in Section 5.2 of the main text shows the gain is dominated by projection/ordering contract conformance, not row-content retrieval
MA-SQLGrid reduces context relative to full-schema-values prompting	C4 uses 259.2 estimated template tokens versus 710.3 for C2 (63.5% smaller) while improving strict accuracy from 0.4389 to 0.7000; measured API input falls from 6346.7 to 4859.0 tokens per question (23.4%)	Template tokens are runner estimates; the measured saving is smaller because a fixed serving-side overhead is shared by all conditions
Validation improves the final result modestly	C5 reaches 131/180 and 0.7278; C5 vs C4 has 12 C5-only and 7 C4-only correct cases	C5 over C4 is small and not a strong standalone superiority claim; latency increases to 5562.5 ms
Answer-shape hints materially affect C4 generation	Removing C4 shape hints lowers execution accuracy from 126/180 to 77/180 and answer-shape accuracy from 0.8889 to 0.4389	Same fixed GridDB test split and generator; only the shape-hint channel is removed
Value hints add a smaller prompt-side gain	Removing C4 value hints lowers execution accuracy from 126/180 to 118/180	One no-value prediction had a provider/extraction failure and was counted as an error; no cross-domain claim is made
The result is evidence-faithful and leakage-controlled	900 predictions/scores, zero contract errors, zero unsafe SQL, zero provider/extraction errors, and zero leakage-scan problems	Does not imply production readiness or external benchmark transfer
The main effects replicate on a second independent generator	`deepseek-chat` with byte-identical prompts: C4 over C2 +0.3611 strict (0.3389 to 0.7000), C5 over C4 +0.0667 (0.7667); projection-tolerant reversal replicates (C2 0.8500 versus C4 0.7944); direct-endpoint input-token reduction 74.7%	Two generator families only (one closed, one open-weight); aggregate-level comparison with no per-question cross-model significance test; C5 stage reimplemented from specification (98.8% selection agreement with the archive)
The statistical interpretation is bounded	Paired sign tests compare per-question outcomes on 180 test items; seed 0 is a deterministic pass label; three temperature-0 repeats of C4/C5 on `deepseek-chat` show a maximum accuracy spread of 1.1 points and 98.3% per-question verdict agreement	Run-to-run stability is measured only for the second generator; no multi-seed study exists for the original generator
The compact-context and validation gains persist at tenfold database scale with bounded input tokens	On the x10 database (10x rows, two distractor tables, byte-identical 180 questions), `deepseek-chat` gives C4 0.5944 versus C2 0.3333 strict (+26.1 points) and C5 0.6500 (+5.6); C4 input tokens stay flat (509.3 to 504.0 per question) versus +51% growth for C2; rebuilt contexts verified byte-identical to the archive on v0.1 (180/180)	Second generator only (original serving stack unavailable); the C4/C5 absolute drop from v0.1 is entirely attributable to the `LIMIT 80` value-inventory truncation (byte-identical prompts 101/149 versus 102/149; changed prompts 6/31 versus 23/31); distractor tables stress only C2 because the C4/C5 builders use a fixed eight-table catalog (Section S4)

S4 Scale Robustness at Tenfold Database Size

This section documents the scale-robustness experiment summarized in Section 5.5 of the main text. A deterministic tenfold-scaled variant of the database (`griddb_maintenance_v2_x10`) was built: every entity table receives ten times the rows (for example, 180 assets instead of 18), and two distractor tables (`vegetation_inspections`, `spare_parts_inventory`) are added to the schema. The 180 test questions and their gold SQL are byte-identical to v0.1, and all 200 gold queries were re-validated against the scaled database before any model call (zero gold-validation errors). Because the original `gpt-5.4-mini` serving stack is no longer available, the experiment measures scale robustness on the second generator: C2, C4, and C5 were re-run with `deepseek-chat` at temperature 0 (540 primary calls plus 13 bounded repair calls, zero provider errors, safe-SQL rate 1.0 in all conditions) and compared against the same generator's v0.1 run. Condition contexts were rebuilt with the original context-builder modules received from the experiment machine after the initial artifact release; as a fidelity check, on the v0.1 database those builders reproduce the archived formal-run prompts byte-for-byte (180/180 for both C2 and C4).

Table S8: Full metric comparison for the scale-robustness run (`deepseek-chat`, byte-identical 180-question test split). Strict: paper primary metric. Order-insensitive: multiset row comparison, exact projection width. Projection-tolerant: order-insensitive and tolerant to column permutation/subset, as in the main-text convention-sensitivity analysis.

Condition	Strict v0.1	Strict x10	Order-insens. v0.1	Order-insens. x10	Proj.-tolerant v0.1	Proj.-tolerant x10
C2	0.3389 (61/180)	0.3333 (60/180)	0.3667	0.3667	0.8500	0.8167
C4	0.7000 (126/180)	0.5944 (107/180)	0.7944	0.6944	0.7944	0.6944
C5	0.7667 (138/180)	0.6500 (117/180)	0.8278	0.7000	0.8278	0.7000

Table S9: Token economics at scale (`deepseek-chat`, provider-reported means per question; C5 includes repair calls when triggered).

Condition	Input tokens v0.1	Input tokens x10	Growth	Output tokens x10	Latency ms x10
C2	2011.6	3042.6	+51%	48.6	989.7
C4	509.3	504.0	−1% (flat)	47.3	1115.9
C5	680.5	560.5	−18%	180.3	1834.1

Table S8 shows that the compact-context advantage persists at +26.1 points strict (C4 0.5944 versus C2 0.3333; +36.1 at v0.1) and that validation still adds +5.6 points (versus +6.7 at v0.1). The convention-sensitivity pattern also persists: 99 of C2's 120 strict errors are shape mismatches, so under the projection-tolerant reading C2 again overtakes the compact conditions (0.8167 versus 0.6944/0.7000), exactly as at v0.1; C4/C5 gain from order-insensitivity but nothing further from projection tolerance, also as at v0.1. Table S9 shows the token divergence: the C2 full-schema-values prompt grew +51% in measured input tokens and must keep growing with database size (it enumerates the schema DDL and the full value dictionary), whereas the C4 compact context stayed flat, making compact grounding roughly six times cheaper in input tokens at the x10 scale. The builder-side context estimates diverge the same way: the C2 context grows from 631 to 1125 estimated tokens (+78%) while the C4 context stays at 174 to 176.

Value-inventory truncation artifact (complete diagnosis). The absolute C4/C5 drop from v0.1 is entirely attributable to a fixed-cap value-inventory truncation, localized as follows. The pilot

builder's value inventory enumerates `SELECT DISTINCT <col> ... ORDER BY <col> LIMIT 80` per column. At v0.1 all 18 asset names fit within the cap; at x10 there are 180 distinct `asset_name` values and the alphabetical top-80 cuts off before the RL-*/SB-*/TX-* names, including the TX-001 to TX-004 literals referenced by test questions, so the affected questions lose their matched-value lines and normalization hints. The decomposition is exact: 149 of the 180 C4 prompts are byte-identical between v0.1 and x10, and on those the x10 run scores 101/149 against 102/149 for the v0.1 predictions re-scored on the x10 database (no scale effect); the entire drop sits in the 31 changed-prompt questions, which fall from 23/31 to 6/31 correct. Two corroborating observations complete the picture. First, C5's repair trigger fires less often at x10 (13 versus 52 repairs) because fewer inferred value hints produce fewer missing-value-hint flags, a downstream symptom of the same truncation. Second, re-scoring the v0.1 predictions on the x10 database moves C4 only from 0.7000 to 0.6944 and leaves C5 at 0.7667, so the v0.1 numbers were not inflated by coincidental denotation matches on tiny tables.

Boundaries. The two distractor tables stress only C2, whose prompt includes their DDL and values: the C4/C5 builders enumerate a fixed eight-table catalog, so the distractors are excluded from the compact context by construction, not by learned schema selection. Part of C4's token boundedness comes from the same `LIMIT 80` cap that causes the accuracy artifact, so the compact context is bounded partly at the cost of value recall. A deployment-grade version should replace the fixed-cap alphabetical value scan with a scalable value index (for example CHESS-style keyword or hash-based value retrieval) once a column exceeds roughly 80 distinct values.